

这份讲义回答的核心问题是：**一个大语言模型预训练完成后，为什么还需要“适配”到具体任务，以及有哪些适配方法。**简单说，预训练模型像一个“通才学生”，已经读过很多书，但真正进入某个岗位、某个业务场景、某个专业任务时，还需要进一步训练或调整，才能稳定完成任务。**Adaptation**，就是把一个通用大模型，针对具体下游任务、领域知识或用户需求进行再调整，让它从“会语言”变成“会做事”。

1. 为什么需要 Adaptation?

讲义一开始强调：仅靠 prompting 或 in-context learning，模型已经可以做一些任务，但不是所有任务都能靠提示词解决。

因为下游任务和预训练任务之间经常存在差异。

1.1 格式不同

预训练时，语言模型主要学的是：

给定前文，预测下一个 token。

但下游任务可能是：

判断两个句子是否蕴含；
回答一个问题；
把表格转成文字；
解析电子病历；
做分类、抽取、摘要。

比如自然语言推理 NLI：

```
Premise: I have never seen an apple that is not red.  
Hypothesis: I have never seen an apple.  
Correct output: Not entailment
```

这不是一个自然的“续写任务”，而是一个判断任务。模型虽然懂语言，但不一定天然适应这种输入输出格式。

通俗地说：

预训练像让模型大量读书，适配像教它考试题型。
会读书不等于会做所有题型。

1.2 主题不同

模型预训练数据可能来自通用互联网文本，比如 The Pile。

但下游任务可能来自非常专业的领域，例如：

- 医疗记录；
- 金融报告；
- 法律合同；
- 企业内部文档；
- 科研论文。

这些内容和通用文本的语言风格、专业词汇、判断标准都不同。

通俗地说：

一个读过很多百科和网页的人，不一定立刻能读懂券商资管产品周报、医院 EHR 病历或法律条款。

1.3 时间不同

模型预训练数据有时间截止点。

如果新知识发生在预训练之后，模型就不知道。例如讲义举例说 GPT-3 训练时还不知道 Biden 成为美国总统。

另外，有些知识不是公开的，比如企业内部知识、客户数据、业务规则，预训练模型天然无法接触。

所以 Adaptation 的另一个目的，是让模型获得：

新知识、领域知识、任务知识、组织内部知识。

2. Adaptation 的基本形式

讲义把适配问题抽象成：

已有一个预训练语言模型参数 θ_{LM} ，现在有一个下游任务数据集：

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

目标是找到一组新的参数 y ，让模型在这个任务上的 loss 更低。

这句话可以翻译成大白话：

我们已经有一个大模型，现在给它一批具体任务样本，让它学会在这个任务上输出更正确的答案。

这些“要训练的参数”可以有不同范围：

- 1. 只训练一个很小的预测头；
- 2. 训练整个大模型；
- 3. 冻结大模型，只训练一些轻量参数。

这也对应讲义的三类核心方法：

方法	冻结什么	训练什么	成本	表达能力
Probing	冻结语言模型	只训练预测头	最低	最弱
Fine-tuning	不冻结	训练整个模型和任务头	最高	最强
Lightweight fine-tuning	冻结大部分模型	训练少量新增参数	较低	较强

3. Probing：把大模型当作“特征提取器”

Probing 的做法是：

冻结原来的语言模型，只在它上面加一个很小的预测头，比如线性分类器或浅层神经网络。也就是说，大模型本身不变，只训练一个小模块来读懂模型内部表示。

通俗解释

可以把大模型想象成一个已经训练好的“文本理解器”。

Probing 就像：

不改变这个理解器，只在它后面接一个小判卷器。
这个判卷器根据模型输出的表示，判断句子类别、词性、情感、蕴含关系等。

比如：

- 输入一句话；
 - BERT 产生每个 token 的 embedding；
 - 我们取一个固定长度的向量；
 - 预测头根据这个向量输出分类结果。
-

3.1 CLS token 和平均池化

讲义还解释了一个关键问题：

Transformer encoder 输入一串 token，会输出一串 embedding。
但很多任务只需要一个结果，比如分类。

那怎么把多个 token embedding 变成一个向量？

常见方法有两个：

1. CLS token

在输入前面加一个特殊 token [CLS]，最后用这个 token 的 embedding 代表整句话。

2. Average over tokens

把所有 token 的 embedding 做平均，得到一个整体表示。

通俗地说：

一篇文章有很多词，每个词都有一个理解向量。

但分类任务只需要一个“整篇文章的代表向量”。

CLS 就像让模型专门安排一个“班长”来代表整句话；平均池化就是让所有 token 投票取平均。

3.2 Probing 的优缺点

优点：

- 成本低；
- 训练快；
- 可以用来分析模型内部表示；
- 适合观察模型是否已经“存储”某些语言知识，比如词性信息。

缺点：

- 模型主体不更新；

- 表达能力有限；
- 如果预训练模型本身没有形成足够好的表示，probe 也很难补救。

所以 Probing 更像：

用来检测模型已经会什么，而不是彻底教会它新任务。

4. Fine-tuning：把整个模型拿到下游任务上继续训练

Fine-tuning 是最传统、最直接的适配方式。

做法是：

把预训练模型参数作为初始化，然后在下游任务数据上继续训练整个模型。

讲义强调，fine-tuning 会优化：

- 语言模型的全部参数；
- 新增的任务预测头参数。

并且预训练阶段的 optimizer state 通常会被丢弃。Fine-tuning 的学习率通常比预训练小一个数量级以上，训练时间也远短于预训练。

通俗解释

预训练像本科通识教育。

Fine-tuning 像进入某个专业后继续深造。

例如：

- 一个通用模型读过很多网页；
 - 现在你给它几万条医疗问答；
 - 它继续训练后，就更适合医疗 QA；
 - 但这个模型会变成“医疗专用版本”。
-

4.1 Fine-tuning 的优缺点

优点：

- 表达能力强；
- 可以深度改变模型行为；

- 通常比 probing 效果好。

缺点：

- 成本高；
- 每个任务都可能需要保存一份完整模型；
- 如果模型很大，存储和部署压力巨大；
- 在小数据集上可能过拟合；
- 可能损伤原有能力。

通俗地说：

Fine-tuning 很强，但很贵。

它像为每个业务场景单独训练一个专属员工，效果好，但维护成本高。

5. Fine-tuning for zero-shot：让模型更会“看懂任务说明”

讲义提到 FLAN 和 T0。

它们的核心思想是：

把很多不同任务统一成 prompt 格式，然后对模型做多任务 fine-tuning，让模型学会根据任务说明完成任务。

这和普通 fine-tuning 的区别在于：

普通 fine-tuning 是让模型适应一个具体任务。

FLAN / T0 是让模型适应“任务说明这种格式”。

通俗地说：

不是只教模型做一道题，而是教模型看懂题干。

这样以后遇到没见过的新题，只要题干写清楚，模型也更可能做对。

这就是为什么这类方法可以提升 zero-shot 能力。

6. InstructGPT：让模型更符合人类偏好

讲义也提到 InstructGPT，也就是后来 RLHF 思路的重要代表。

它的目标不是单纯让模型预测下一个 token，而是让模型输出更符合人类需求的答案：

- helpful：有帮助；
- honest：诚实；
- harmless：无害。

语言建模本身并不天然保证这些目标。

因为预训练目标只是：

根据前文预测最可能出现的下一个词。

但“最可能出现”不等于“最有帮助、最诚实、最安全”。

6.1 InstructGPT 的三步

讲义概括为三步：

1. 收集人工示范数据

人类写出理想回答，用这些数据做 supervised fine-tuning。

2. 收集人类偏好数据

对同一个指令，让模型生成多个回答，请人类排序哪个更好。

3. 用强化学习优化模型

训练奖励模型，再让语言模型最大化人类偏好奖励。

通俗地说：

第一步：老师亲自示范怎么答。

第二步：老师比较几个学生答案，告诉你哪个更好。

第三步：学生不断练习，目标是获得老师更高评价。

讲义中还提到一个很有代表性的结果：1.3B 的 InstructGPT 在很多时候比 175B 的 GPT-3 更受人类偏好。这说明：

模型不是越大就越好，是否经过适配和对齐也非常关键。

7. Lightweight Fine-tuning：参数高效微调

Fine-tuning 效果好，但太贵。

Probing 成本低，但能力有限。

所以讲义引出第三类方法：

Lightweight fine-tuning，也就是轻量微调 / 参数高效微调。

它的目标是：

尽量接近 full fine-tuning 的表达能力，但不需要为每个任务保存完整大模型。

核心思想是：

冻结大部分模型，只训练很少量新增参数，通常少于模型总参数的 1%。

这类方法包括：

- prompt tuning；
- prefix tuning；
- adapter tuning；
- LoRA；
- BitFit 等。

讲义重点讲了前三种。

8. Prompt tuning：训练“软提示词”

Prompt tuning 的做法是：

在输入前面加一串可学习的连续向量，也就是 soft prompt，然后只训练这些 prompt embedding，冻结整个语言模型。

注意，这里的 prompt 不是人工写出来的自然语言提示词，而是一组模型可学习的向量。

通俗解释

普通 prompt engineering 是人写提示词：

请判断下面两句话是否蕴含：
...

Prompt tuning 则是：

不让人写提示词，而是让模型自己学出一串“看不懂但有用”的向量提示。

这些向量对人来说没有自然语言含义，但对模型来说可能非常有效。

讲义还提到，模型越大，prompt tuning 越接近 full fine-tuning 的效果。也就是说：

大模型本身能力越强，只需要轻轻“拨一下方向”，就能适配任务。

9. Prefix tuning：在每一层 Attention 里加可学习前缀

Prefix tuning 比 prompt tuning 更深入一点。

Prompt tuning 只在输入层加 soft prompt。

Prefix tuning 则是在每一层 attention 的 key 和 value 上加可学习参数。

讲义用 attention 公式解释了这个机制。

简单说：

Prefix tuning 不只是给输入前面加提示，而是在模型每一层思考时，都塞入一些可学习的“上下文记忆”。

通俗解释

Prompt tuning 像是在开会前给模型一张提示纸。

Prefix tuning 像是在模型每一轮思考过程中，都给它一个隐形助手提醒方向。

所以 prefix tuning 对生成任务，比如摘要、生成、风格迁移等，常常很有用。

10. Adapter tuning：在 Transformer 层之间插入小模块

Adapter tuning 的做法是：

在每个冻结的 Transformer 层之间插入一个小的可训练 adapter 模块。

这些 adapter 通常是一个瓶颈结构：

先降维 → 非线性变换 → 再升维 → 加回原表示

讲义中的形式是：

$$\text{Adapter}(x) = x + W_{up} \sigma(W_{down} x)$$

通俗地说：

原来的大模型不动。

但在每一层之间插入一个小插件。

这个插件负责把模型的行为稍微调整到当前任务需要的方向。

这很像给一个大型软件安装插件：

- 主程序不改；
 - 插件很小；
 - 不同任务装不同插件；
 - 存储和部署成本比复制整个软件低很多。
-

11. 轻量微调为什么重要？

讲义强调轻量微调的核心价值是：

对每个任务或用户，只保存少量参数，而不用保存完整大模型。

例如，如果要为很多用户部署个性化模型：

- full fine-tuning：每个用户一份完整模型，成本极高；
- prefix tuning：每个用户只保存一份 prefix，小很多；
- 推理时把对应用户的 prefix 加到输入上即可。

通俗地说：

不是给每个人配一台完整机器，而是大家共用一台大机器，每个人带自己的小配置文件。

这对大模型实际落地非常关键。

12. 轻量微调 and 泛化能力

讲义还提到一个有意思的现象：

Lightweight fine-tuning 往往比 full fine-tuning 在 OOD，也就是分布外测试上更稳健。

例如：

- 在 SQuAD 上训练，在 MRQA 其他领域任务上测试；
- 在新闻摘要上训练，在体育新闻或其他新闻类别上测试。

原因可以这样理解：

Full fine-tuning 会深度改动模型，可能过度贴合训练任务。

轻量微调只调整少量参数，更像是在保留通用能力的基础上做方向引导，所以有时更不容易过拟合。

当然，讲义也提醒：如果测试集和训练集同分布，full fine-tuning 的效果通常还是略好。

13. 三类方法的本质区别

可以把这三类方法理解成三个层级。

13.1 Probing：只训练一个小头

大模型不动，只在上面接一个判断器。

适合：

- 表征分析；
- 简单分类；
- 低成本任务；
- 检查模型是否已经学到某类信息。

缺点是能力有限。

13.2 Fine-tuning：整个模型都继续训练

把大模型整体拿到具体任务上继续训练。

适合：

- 数据较多；
- 任务很重要；
- 追求最佳效果；
- 资源允许保存专用模型。

缺点是贵、重、难维护。

13.3 Lightweight fine-tuning：只训练少量关键参数

大模型大部分冻结，只训练 prompt、prefix、adapter、LoRA 等小参数。

适合：

- 多任务；
- 多用户；
- 个性化；
- 成本敏感；
- 想兼顾效果和部署效率。

这是现在大模型应用中非常核心的一类方法。

14. 用一个类比理解整份讲义

你可以把大模型想象成一个已经完成通识教育的优秀毕业生。

Prompting

你只是临时给他一句指令：

“请你帮我做这个任务。”

适合简单任务，但复杂任务不稳定。

Probing

你不训练他本人，只给他配一个小助理：

“你负责理解材料，小助理负责输出分类结果。”

成本低，但能力有限。

Fine-tuning

你让他重新参加专业培训：

“以后你就是医疗问答专家 / 金融报告专家 / 法律合同专家。”

效果强，但成本高。

Lightweight fine-tuning

你不给他重新培训全部知识，而是给他装一个专业插件：

“原来的能力保留，只加一个小模块，让他更适合当前任务。”

成本低，效果也不错。

15. 启发

大模型能力不是只来自预训练，真正落地时，Adaptation 决定了模型能否进入具体业务场景。

因为真实业务里，模型往往面对的是：

- 金融术语；
- 内部考核口径；
- 公司业务规则；
- 非公开数据；
- 多部门协作语境；
- 复杂报告格式；
- 特定输出模板。

这些都不是靠通用预训练天然解决的。

所以实际 AI 系统建设里经常要考虑：

1. 能不能只靠 prompt；
2. 是否需要 RAG 补充知识；
3. 是否需要 fine-tuning；
4. 是否可以用 LoRA / adapter / prompt tuning；
5. 是否要做 instruction tuning；
6. 是否需要人类偏好数据来对齐输出风格。

从工程视角看，Adaptation 是从“模型能力”走向“业务可用性”的关键桥梁。